암호화 알고리즘

개요

CometBFT는 블록체인의 보안과 무결성을 보장하기 위해 다양한 암호화 알고리즘을 사용합니다. 이 문서에서는 CometBFT에서 사용되는 주요 암호화 알고리즘들의 특성과 선택 이유를 상세히 설명합니다.

CometBFT의 암호화 시스템은 다음과 같은 핵심 요소들로 구성됩니다:

- **해시 함수**: 데이터 무결성과 식별자 생성
- **디지털 서명**: 트랜잭션과 블록의 인증
- **주소 생성**: 계정과 밸리데이터 식별
- **머클 트리**: 효율적인 데이터 검증

핵심 개념

암호화 프리미티브 (Cryptographic Primitives)

CometBFT에서 사용되는 기본적인 암호화 구성 요소들:

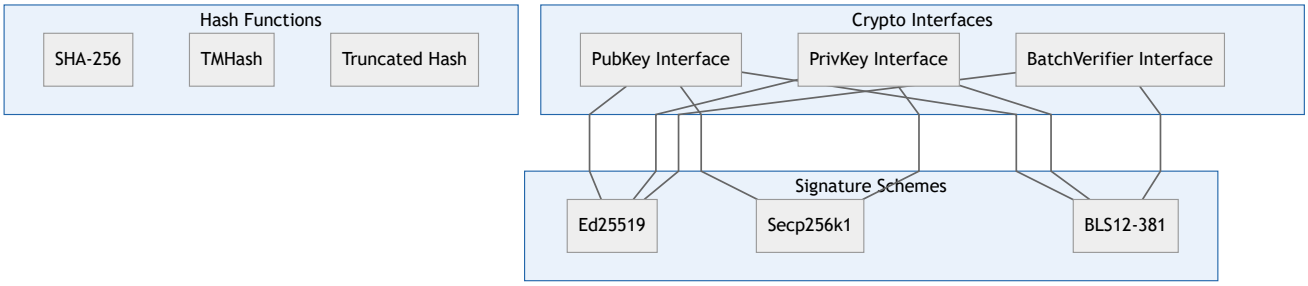
- **해시 함수**: 임의 크기의 데이터를 고정 크기의 해시값으로 변환
- **디지털 서명**: 메시지의 진위성과 무결성을 보장
- **공개키 암호화**: 비대칭 키 쌍을 이용한 보안 통신
- **머클 증명**: 대용량 데이터셋의 효율적인 검증

보안 요구사항

- **충돌 저항성**: 동일한 해시값을 가지는 서로 다른 입력을 찾기 어려움
- **위조 불가능성**: 개인키 없이는 유효한 서명을 생성할 수 없음
- **검증 가능성**: 공개키로 서명의 유효성을 확인할 수 있음
- **결정론적**: 동일한 입력에 대해 항상 동일한 출력 생성

아키텍처

암호화 인터페이스 구조



암호화 컴포넌트 계층

```
// 기본 암호화 인터페이스
type PubKey interface {
Address() Address
Bytes() []byte
VerifySignature(msg []byte, sig []byte) bool
Type() string
}
```

```go
type PrivKey interface {
Bytes() []byte
Sign(msg []byte) ([]byte, error)
PubKey() PubKey
Type() string
}
```

해시 함수

SHA-256

CometBFT의 기본 해시 함수로 SHA-256을 사용합니다.

특성

- **출력 크기**: 256비트 (32바이트)
- **블록 크기**: 512비트 (64바이트)
- **보안 강도**: 128비트 (2^128 연산 필요)
- **표준**: NIST FIPS 180-4

사용 용도

```go
// 기본 해시 계산
func  Sha256(bytes []byte) []byte {
hasher := sha256.New()
hasher.Write(bytes)
```

```go
return hasher.Sum(nil)
}

// 다중 데이터 해시
func SumMany(data []byte, rest ...[]byte) []byte {
h := sha256.New()
h.Write(data)
for _, data := range rest {
h.Write(data)
}
return h.Sum(nil)
}
```

선택 이유

1. **검증된 보안성**: 광범위한 암호학적 분석을 통과
2. **표준화**: 국제 표준으로 채택되어 상호 운용성 보장
3. **성능**: 하드웨어 가속 지원으로 빠른 연산 속도
4. **비트코인 호환성**: 기존 블록체인 생태계와의 호환성

TMHash (Truncated Hash)

CometBFT에서 주소 생성을 위해 사용하는 단축된 해시입니다.

특성

- **기반**: SHA-256

- **출력 크기**: 160비트 (20바이트)
- **용도**: 주소 생성, 식별자 생성

구현

```
const TruncatedSize = 20

// 단축된 해시 생성
func SumTruncated(bz []byte) []byte {
hash := sha256.Sum256(bz)
return hash[:TruncatedSize]
}

// 주소 생성
func AddressHash(bz []byte) Address {
return Address(tmhash.SumTruncated(bz))
}
```

장점

- **공간 효율성**: 주소 크기 최소화
- **충돌 저항성**: 2^80 보안 강도 제공
- **이더리움 호환성**: 20바이트 주소 형식 사용

디지털 서명 알고리즘

Ed25519

CometBFT의 기본 서명 알고리즘입니다.

특성

- **곡선**: Curve25519 (Edwards 형태)
- **키 크기**: 공개키 32바이트, 개인키 64바이트
- **서명 크기**: 64바이트
- **보안 강도**: 128비트

구현 세부사항

```go
const (
PubKeySize = 32
PrivateKeySize = 64
SignatureSize = 64
SeedSize = 32
)

// 서명 생성
func (privKey PrivKey) Sign(msg []byte) ([]byte, error) {
signatureBytes := ed25519.Sign(ed25519.PrivateKey(privKey), msg)
return  signatureBytes, nil
}
```

```go
// 서명 검증
func (pubKey PubKey) VerifySignature(msg []byte, sig []byte) bool {
if len(sig) != SignatureSize {
return false
}
return cachingVerifier.VerifyWithOptions(
ed25519.PublicKey(pubKey), msg, sig, verifyOptions)
}
```

선택 이유

1. **성능**: 빠른 서명 생성과 검증
2. **보안성**: 사이드 채널 공격에 대한 저항성
3. **결정론적**: 동일한 메시지에 대해 동일한 서명 생성
4. **ZIP-215 호환성**: 확장된 검증 규칙 지원

배치 검증 지원

```go
type BatchVerifier struct {
*ed25519.BatchVerifier
}

// 다중 서명 검증
func (b *BatchVerifier) Verify() (bool, []bool) {
return b.BatchVerifier.Verify(crypto.CReader())
```

```
}
```

Secp256k1

비트코인과 호환성을 위해 지원되는 서명 알고리즘입니다.

특성

- **곡선**: secp256k1 (Koblitz 곡선)
- **키 크기**: 개인키 32바이트, 공개키 33바이트 (압축)
- **서명 크기**: 64바이트
- **보안 강도**: 128비트

구현 특징

```
const (
PrivKeySize = 32
PubKeySize = 33  // 압축된 공개키
)

// 서명 생성 (낮은 S 값 강제)
func (privKey PrivKey) Sign(msg []byte) ([]byte, error) {
priv := secp256k1.PrivKeyFromBytes(privKey)
sum := sha256.Sum256(msg)
sig := ecdsa.SignCompact(priv, sum[:], false)
return  sig[1:], nil  // 복구 바이트 제거
```

```
}
```

주소 생성

```go
// 비트코인 스타일 주소: RIPEMD160(SHA256(pubkey))
func (pubKey PubKey) Address() crypto.Address {
hasherSHA256 := sha256.New()
hasherSHA256.Write(pubKey)
sha := hasherSHA256.Sum(nil)
hasherRIPEMD160 := ripemd160.New()
hasherRIPEMD160.Write(sha)
return  crypto.Address(hasherRIPEMD160.Sum(nil))
}
```

사용 사례

- **비트코인 호환성**: 기존 비트코인 지갑과의 호환
- **레거시 지원**: 기존 시스템과의 연동
- **다중 서명**: 복잡한 서명 스킴 구현

BLS12-381

집계 서명을 위한 고급 서명 알고리즘입니다.

특성

- **곡선**: BLS12-381 (페어링 친화적)
- **키 크기**: 개인키 32바이트, 공개키 96바이트

- **서명 크기**: 96바이트
- **보안 강도**: 128비트

핵심 기능

```
// 서명 생성
func (privKey PrivKey) Sign(msg []byte) ([]byte, error) {
signature := new(blstSignature).Sign(privKey.sk, msg, dstMinPk)
return signature.Compress(), nil
}

// 서명 검증
func (pubKey PubKey) VerifySignature(msg, sig []byte) bool {
signature := new(blstSignature).Uncompress(sig)
if signature == nil {
return false
}
return signature.Verify(false, pubKey.pk, false, msg, dstMinPk)
}
```

집계 서명 장점

1. **공간 효율성**: 다중 서명을 하나로 집계
2. **검증 효율성**: 배치 검증 지원
3. **확장성**: 대규모 밸리데이터 세트 지원

4. **이더리움 2.0 호환성**: 표준 BLS 구현

머클 트리

RFC-6962 호환 구현

CometBFT는 Certificate Transparency 표준을 따르는 머클 트리를 구현합니다.

구조

```
// 바이트 슬라이스로부터 머클 루트 계산
func HashFromByteSlices(items [][]byte) []byte {
return hashFromByteSlices(sha256.New(), items)
}

// 재귀적 머클 트리 구성
func hashFromByteSlices(sha hash.Hash, items [][]byte) []byte {
switch len(items) {
case 0:
return emptyHash()
case 1:
return leafHashOpt(sha, items[0])
default:
k := getSplitPoint(int64(len(items)))
left := hashFromByteSlices(sha, items[:k])
right := hashFromByteSlices(sha, items[k:])
```

```go
return innerHashOpt(sha, left, right)
}
}
```

분할 전략

```go
// 2의 거듭제곱으로 균형 잡힌 트리 생성
func getSplitPoint(length int64) int64 {
if length < 1 {
panic("Trying to split a tree with size < 1")
}
uLength := uint(length)
bitlen := bits.Len(uLength)
k := int64(1 << uint(bitlen-1))
if k == length {
k >>= 1
}
return k
}
```

성능 최적화

- **반복적 구현**: 스택 오버플로우 방지
- **균형 트리**: 최적의 검증 경로 길이

- **캐싱**: 중간 해시값 재사용

성능 고려사항

해시 함수 최적화

1. **하드웨어 가속**: SHA-256 전용 명령어 활용
2. **배치 처리**: 다중 해시 동시 계산
3. **메모리 효율성**: 스트리밍 해시 계산

서명 검증 최적화

1. **배치 검증**: Ed25519와 BLS12-381 지원
2. **캐싱**: 공개키 확장 형태 캐시
3. **병렬 처리**: 독립적인 서명 병렬 검증

메모리 관리

```
// 캐싱 검증기 (Ed25519)
var  cachingVerifier = cache.NewVerifier(cache.NewLRUCache(cacheSize))


// 배치 검증기 풀링
type BatchVerifier struct {
*ed25519.BatchVerifier
}
```

보안 고려사항

난수 생성

```go
// 암호학적으로 안전한 난수 생성기
func CReader() io.Reader {
return rand.Reader
}

// 키 생성에서의 엔트로피 확보
func GenPrivKey() PrivKey {
return genPrivKey(crypto.CReader())
}
```

키 검증

```go
// BLS 공개키 부분군 검사
func (pk *blstPublicKey) KeyValidate() bool {
// 무한원점 및 부분군 검사
return pk.KeyValidate()
}

// Ed25519 키 초기화 검증
func (privKey PrivKey) PubKey() crypto.PubKey {
```

```go
initialized := false
for _, v := range privKey[32:] {
if v != 0 {
initialized = true
break
}
}
if !initialized {
panic("Expected ed25519 PrivKey to include concatenated pubkey bytes")
}
// ...
}
```

서명 검증 강화

```go
// Secp256k1 가단성 방지
func (pubKey PubKey) VerifySignature(msg []byte, sigStr []byte) bool {
// 낮은 S 값 강제 (가단성 방지)
modifiedSignature, parseErr := ecdsa.ParseDERSignature(signature.Serialize())
if parseErr != nil {
return false
}
if !signature.IsEqual(modifiedSignature) {
```

```
return false

}

// ...

}
```

상호 운용성

표준 준수

1. **NIST 표준**: SHA-256 (FIPS 180-4)
2. **RFC 표준**: Ed25519 (RFC 8032), 머클 트리 (RFC 6962)
3. **이더리움 호환성**: BLS12-381, 20바이트 주소
4. **비트코인 호환성**: Secp256k1, RIPEMD160 주소

크로스 체인 지원

- **다중 서명 알고리즘**: 다양한 블록체인과의 호환성
- **표준 주소 형식**: 기존 지갑과의 호환성
- **표준 해시 함수**: 범용적인 검증 가능성

이러한 암호화 알고리즘들의 조합을 통해 CometBFT는 높은 보안성과 성능을 동시에 달성하며, 다양한 블록체인 생태계와의 호환성을 보장합니다.